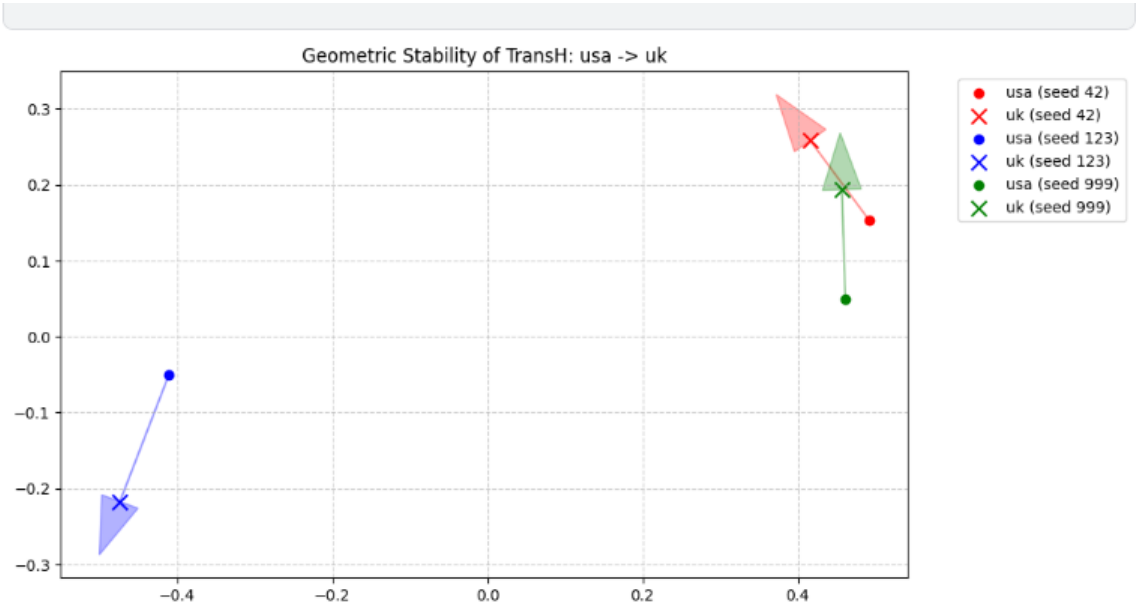
Understanding The Pykeen routing decisions

```

TransH_s42_e100: rank=1, predicted='uk', magnitude=0.000

TransH_s123_e100: rank=5, predicted='usa', magnitude=0.313

TransH_s999_e100: rank=3, predicted='egypt', magnitude=1.035

DistMult_s42_e100: rank=7, predicted='cuba', magnitude=1.690

DistMult_s123_e100: rank=9, predicted='poland', magnitude=1.657

DistMult_s999_e100: rank=13, predicted='cuba', magnitude=1.650

ConvKB_s42_e100: rank=5, predicted='egypt', magnitude=5.988

ConvKB_s123_e100: rank=3, predicted='ussr', magnitude=5.797

ConvKB_s999_e100: rank=3, predicted='netherlands', magnitude=5.543

=== STABILITY ACROSS SEEDS ===

  model epochs stability

 ConvKB     100     0.0682

 TransH     100     0.0616

Dismult     100    -0.0611
```

- TransH is completely inconsistent across the seeds, predicts itself, then predicts usa and then predicts egypt that is not part of its own clusters, seed 42 gets it right but seed 123 and then seed 999 gets them completely wrong for tail predictions, which means that the hyperplane projection has completely collapsed
- TransH has a stability of 0.06, these three failure vectors point in completely different directions. Ne has 0 magnitude and one points towards itself and the other points towards a wrong block, this is not a structural failure but is unstable
- Dismult – consistently wrong
- The magnitudes for all 3 seeds are wrong, they cannot predict by the same x amount, but the stability Is negative meaning they point in directions across even though the magnitude is same
- Dismult is symmetric In nature it cannot distinguish directions, so when it fails symmetrically the error vectors cannot understand the directions hence it is a architecture failure rather than a data failure
- ConvKB magnitudes are enormous compared to the TransH due to the complexity, seed 999 and seed 23 both predicted rank 3 and seed 999 predilected netherlands which is the correct answer in the test set but it is just another answer

- Stability amongst the magnitude this is the highest, that's structural they predicted different tails but structurally they where consistent throughout the seeds
-

- DUE TO THIS IT MAKES SENSE TO USE CONVKB TO MAYBE UNDERSTAND HOW LLMS IN AGENTS' INSTRUCTION TUNED REACT TO THIS FAILURE UNDER THE RGIHT STRUCTURAL ASSUMPTION
- CONVKB has low bias and variance, whereas TransH has low bias but high variance, on the other hand Dismult has high bias and low variance
- Why we choose CONVKB because we want the agents to reason over structural failure nodes, we cannot random correct initialization stochasticity being reasoned over by the Agenet

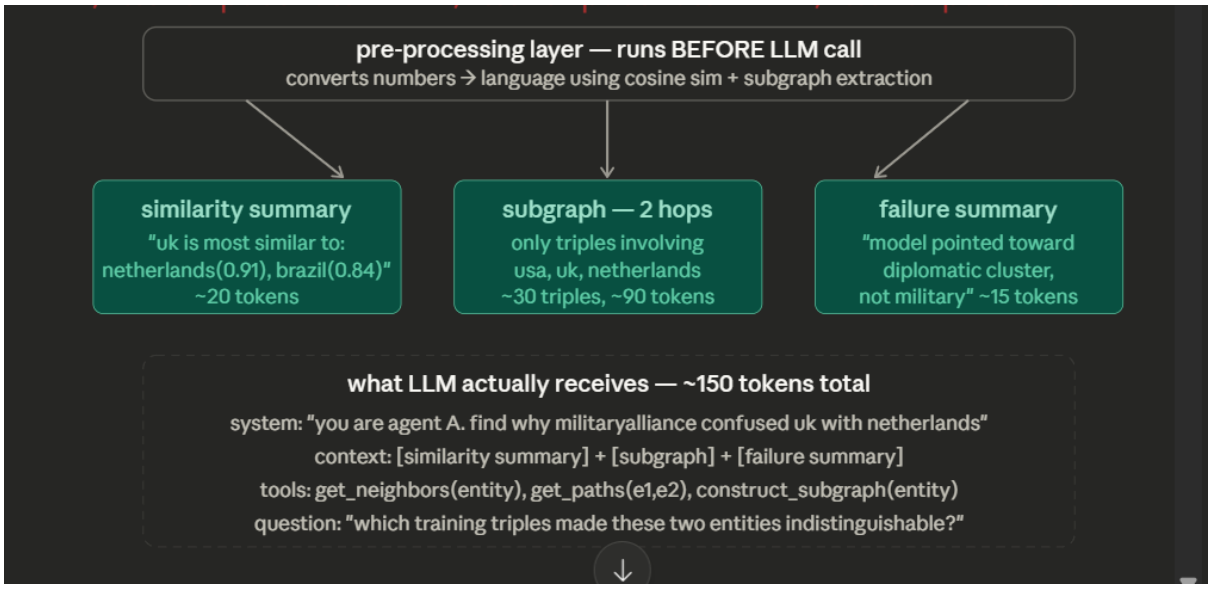


- From the above visualization we see 2 seeds are in the correction direction but 1 seed has completely rotated away from the right tail to be predicted this is called as embedding space rotation – a known problem of the TransE models because the loss function cares about a relative entity not about absolute positions 2 completely different embeddings can produce same loss values, they might be correct mathematically but they produce different predictions

PREPROCESSING LAYER:

Why? Agents cannot directly understand direct embeddings as they have a finite context and raw embeddings are useless to them directly

<u>We need to translate the embeddings into a language that the LLM understand and can reason about</u>



Translation 1 : Embeddings ->similarity -> we compute the top_k_simillar for the training data , the LLM understands the training neighbourhood without seeing a single float

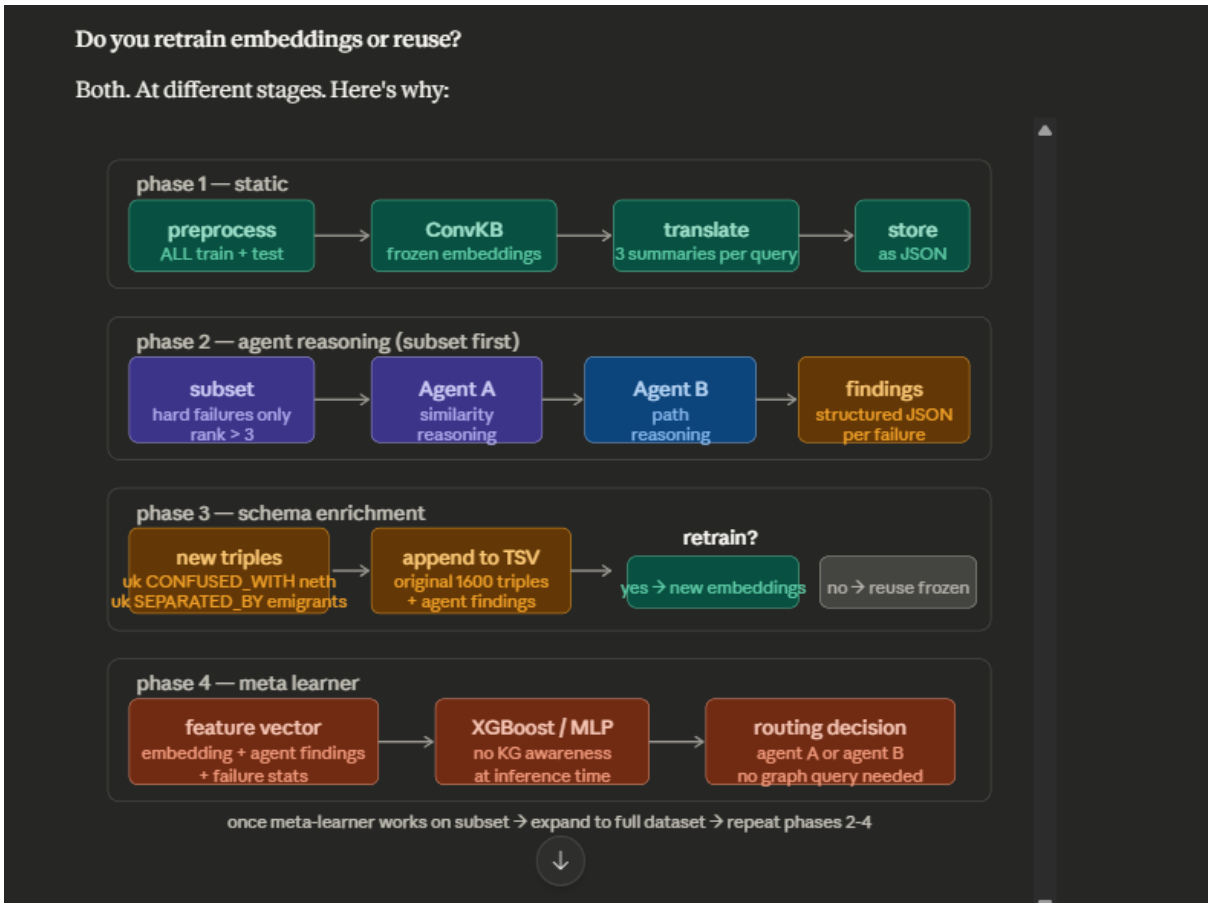
Translation 2: Full schema -> Focused subgraph

- Identify queries that require multihop reasoning if the query entities
- Outputs related queries that it things that the makes sense

Translation 3: Failure vector -> Failure Summary

The big picture of the pipeline:

```
preprocess ALL data (train + test)
      ↓
subset samples for agent reasoning
      ↓
agents diagnose failures → output structured findings
      ↓
findings become new triples in the schema
      ↓
meta-learner trains on enriched schema
      ↓
question: retrain embeddings or reuse?
```



The meta learner is a fixed feature space, if you retrain embeddings every time the schema grows the vector dimensions mean different things each tome, Feature 7 might represent another relationship but on re-training could encode something else differently. The meta learner cannot learn the temporal target hence we make them static

Retrain Checkpoints – when we have accumulated new generated triples justify a full re-training cycles

```
inner loop (fast):
    agent reasons → findings → new triples →
    meta-learner trains on frozen embeddings + findings
    repeat for each failure

outer loop (slow):
    accumulate N new triples from inner loop
    retrain ConvKB on enriched TSV
    new embeddings become the new frozen base
    restart inner loop
```

For Nations tonight — the inner loop runs on 21 test triples. The outer loop triggers when you've generated enough agent findings to meaningfully change the graph structure. With 21 test triples you might get 15-20 new agent-generated triples. That's enough to run one outer loop cycle.

The meta learner sees over 150+ features but never queries the graph, It just read the row and predicts the agent to route it , That’s the distillation – all the graph reasoning happened during the inner loop, now it lives in tabular features